

2026-10-02

tutorialverse: Interoperability and quality infrastructure for R tutorials

Shaurita D. Hutchins 
University of Alabama at Birmingham

Samuel Bharti 
University of Alabama at Birmingham

Executive summary

Choosing an R tutorial often means inspecting its repository to check the requirements and find evidence that the exercises still work. Lessons use different formats and describe their requirements inconsistently. Instructors and learners may also struggle to determine whether anyone still maintains them.

tutorialverse will help instructors and learners find R tutorials suited to their needs, understand the prerequisites, and see when the materials were last checked and reviewed. We will build an R package that checks tutorials through adapters for existing engines and records the results in a common format. A searchable registry will bring those reports together with tutorial descriptions and human reviews. Authors will keep their lessons in their own repositories, with a community process for reviewing submissions and handling outdated entries.

The 12-month project will initially support `learnr`, `learnr2`, and Quarto Live, with four small example tutorials to test the adapters across teaching contexts. Carpentries-style materials will also be indexed, with stated limits on automated checks. Existing tools will provide browser execution and static hosting.

Signatories and community support

Shaurita D. Hutchins and Samuel Bharti are PhD candidates at the University of Alabama at Birmingham and will co-lead the project.

Hutchins will lead educational design and community development. She will coordinate the example tutorials and develop the metadata schema. Her experience in open-source training and Carpentries instruction will inform the accessibility guidance and contribution process she develops. Trained in genetics, genomics, and bioinformatics, she combines computational biology with reproducible R/Bioconductor workflows. Her [Prioritizing Candidate Genetic Variants Using GTEx, HuBMAP, and IDG](#) module for the CFDE Training Center includes coding activities that run in the browser and knowledge checks.

Bharti will lead software architecture and engine integration. He will connect the package to existing engine checks through adapters that produce common validation reports. His teaching work includes [r-for-beginners](#), an adaptation of Software Carpentry lessons with slides. He also develops the R packages `bioclients`, `biohttp`, and `biobouncer`, and publishes package documentation through GitHub Actions and GitHub Pages.

We will jointly design the package API and registry, and share responsibility for governance and the example tutorials.

We requested public feedback through [r/Stats](#) and the [learnr webR discussion](#). Community contributor [@coatless](#) directed us to `learnr2` and Quarto Live, prompting us to focus on shared metadata and checks. This was feedback on related tools; the proposed schema still needs review. We will seek engine maintainers' feedback on the schema and the checks each adapter can support.

We will also consult Carpentries and Bioconductor educators, along with university and community instructors. We will add documented feedback and letters of support as received.

Problem

Learners need tutorials that match their goals and current knowledge; instructors need lessons they can assess and reuse in their teaching. Both need clear information about content, prerequisites, difficulty, and duration. They also need evidence that the exercises still work: a tutorial may remain online after its exercises break or its author stops maintaining it.

learnr runs tutorials as Shiny applications ([Aden-Buie et al. 2025](#)). learnr2 builds on Quarto Live, which uses webR to execute R in the browser ([learnr2](#), n.d.; [Stagg 2026](#)). tutorialverse will give instructors and learners a common way to compare tutorial descriptions and check results across these engines. Carpentries and Bioconductor materials will provide additional teaching contexts.

Evidence of need and prior work

Published comparisons document the tradeoffs among tutorial tools. For his Quarto exercises, Handel ([2025](#)) compared learnr, webexercises, checkdown, exams, quarto-webr, and quarto-live. He assessed hosting and maintenance requirements alongside the supported exercise types. [Kranz’s 2019 comparison](#) of RTutor and learnr shows that choices about exercise structure, teaching environments, and long-term support predate webR.

Discussions in the learnr community document difficulties [integrating custom Shiny questions](#) and a report of [Python exercises working locally but failing after deployment](#). These reports illustrate support needs; they do not establish how often tutorials fail.

Existing projects address discovery and reuse. The [rstats-ed project](#) assembled a user-curated list of R courses and learning materials, while acknowledging that it could not be comprehensive. TeSS aggregates training metadata, and Bioschemas provides shared descriptions for training resources ([Garcia et al. 2020](#); [Castro et al. 2023](#)). Glittr.org helps users find and compare teaching repositories; its 2024 paper explicitly excludes evaluation of their educational quality ([Geest et al. 2024](#)). The Galaxy Training Network provides a precedent for combining automated checks with community review and ongoing maintenance ([Hiltemann et al. 2023](#)).

tutorialverse will build on this work across R tutorial engines. We will map tutorial descriptions to existing standards and link check results to human reviews and maintenance information. Each result will identify the exercises and dependencies tested, together with the environment and date. Existing engines will run the tutorials; a new engine, browser runtime, or learning-management system is outside the project scope. Static hosting will remain a deployment mode to describe and check.

Proposal

Milestone 1: A minimal metadata schema

We will begin with eight required fields: identifier, title, source URL, lesson URL, engine or format, maintainer, license, and learning objectives. We will map these fields to Schema.org and the Bioschemas TrainingMaterial profile where applicable. Topic, level, prerequisites, packages, deployment mode, duration, accessibility information, and software versions will be optional in the initial schema. The checking tools will generate validation dates and results.

Authors will reuse metadata from their tutorial sources and supply missing fields in a small accompanying file, without rewriting lessons. Missing values will be explicit. An adapter can run a check only when its required fields are supplied or reliably inferred. Nominations with unknown maintainers or licenses will remain pending review.

Engine maintainers and educators will be invited to review the draft. The pilot will measure author effort and inform version 1.0 of the schema, published with example records and adoption guidance.

Milestone 2: Adapters for existing engines

Adapters will connect each engine to the common checking interface, translating its metadata and check results into the shared format. Proposed interfaces include `learnr_adapter()`, `learnr2_adapter()`, and

`quarto_adapter()`, with Quarto Live support in the Quarto adapter. We will refine the interfaces with maintainers and document the versions and features each adapter supports.

Adapters may invoke an engine's existing build or checking tools; the engine remains responsible for execution. Conversion between engines is outside the adapters' scope. A capability table will distinguish metadata import, rendering checks, code execution, and accessibility checks. For Carpentries-style materials, common metadata and link checks will support indexing; deeper checks will depend on the lesson's build system. Registry entries will state those limits.

Milestone 3: Validation across engines

`check_tutorial()` will combine common checks with the selected adapter. It will check metadata, links, and required files, then assess declared dependencies and package versions. Where the engine supports it, the adapter will test rendering and executable code and identify missing or duplicate exercise identifiers. Basic automated accessibility checks will accompany human review of keyboard use and instructional clarity.

Every report will identify failing fields or exercises and record the tested environment, source version, and date. Across engines, unsupported or unrun checks will remain distinct from passes and failures. Interactive exercises must be tested directly: a reachable page, a successful render, or precomputed output is not enough. Automated checks alone will not establish teaching quality or full accessibility.

We will publish a reusable GitHub Actions workflow and integration guidance so authors can run checks in their own repositories without joining the registry. We will test the package on Linux, macOS, and Windows, using examples with known faults to verify that the checks detect problems.

Milestone 4: A searchable tutorial registry

The registry will link tutorial descriptions to the original repositories and published lessons. Authors will retain responsibility for content and hosting. YAML entries will feed a searchable website and a published `index.json` through automated builds. A proposed `tutorial_search()` function will return results as an R data frame.

Readers will filter by topic, package, level, learning objective, engine, and deployment mode when those fields are available. Each entry will show duration and maintenance status when known, and distinguish metadata validity from execution checks and human review. Scheduled checks will flag broken links and outdated validation records.

The pilot will index 5–10 existing tutorials across the supported formats, recording missing fields and manual corrections to measure author effort. Completion will require valid records and working search results. Execution claims will require separate adapter tests.

Milestone 5: Review, governance, and example tutorials

Authors will submit entries through structured pull requests, and others will be able to nominate tutorials. Automated checks will precede human review of instructional clarity, accessibility, scope, licensing, and maintenance expectations. Review criteria will draw on open lesson-review practices ([The Carpentries 2022](#)). Published guidance will distinguish nominations from author submissions and explain the process for correcting, removing, or archiving entries. We will also document responsibility for maintaining each part of tutorialverse.

We will use four small example tutorials to test the adapters: `renv`, package development with `usethis` and `devtools`, an introductory DESeq2 workflow, and a Carpentries R companion. We will choose representative exercises and adapt existing material under compatible licenses. Bharti's `r-for-beginners` lessons and Bioconductor training resources offer starting points.

During development, the examples will test metadata mapping and validation reports as well as each engine's checks. Together they will cover all three adapters; each tutorial need not run under every engine. Tests will include browser interaction where supported and local or server execution where needed. Three or four small integration templates will help authors connect existing lessons to tutorialverse through their chosen engine.

Project plan

Preliminary work

We reviewed existing tutorial tools alongside registries and metadata schemas. [Community feedback](#) helped us focus the project on common metadata and checks. Our prior teaching work includes [Hutchins's candidate genetic variants module](#) and [Bharti's r-for-beginners lessons](#).

Schedule and working practices

Period	Work and completion criteria
Months 1–2	Review the eight-field schema, map metadata from existing tutorials into pilot records, establish repositories and governance, and begin the example tutorials.
Months 3–5	Implement the three adapters and <code>check_tutorial()</code> . Publish a capability table and common report format. Test valid examples and known failures.
Months 6–8	Build registry ingestion and search. Publish <code>index.json</code> and the R search interface. Run the pilot records through the package and document author effort.
Months 9–10	Pilot nominations and human review with external contributors. Test stale-entry handling and gather usability feedback.
Months 11–12	Complete example tutorial and integration-template checks. Address high-priority feedback and release version 1.0 of the package, specification, and registry with maintenance guidance.

We will work openly on GitHub, using issues and milestones to track progress and pull requests for code review. We will publish monthly updates and milestone reports and invite community feedback. Package tests will use `testthat`, with `renv` to reproduce software environments. We will publish documentation with `pkgdown` and distribute the package through R-universe where appropriate.

Code will use the MIT license. Newly authored content and documentation will use CC BY 4.0 where compatible with source obligations; adaptations will retain required attribution and licensing. We will share release announcements and integration guidance with R education communities.

Budget and funding

Requested budget: USD 10,000. This funds 200 hours at USD 50/hour, allocated across the five milestones below. No hardware, cloud, travel, or fee costs are included.

Milestone	Hutchins (h)	Bharti (h)	Total (h)	USD
Minimal schema	20	10	30	1,500
Adapters	5	45	50	2,500
Validation package	15	40	55	2,750
Registry	15	20	35	1,750
Review, governance, and examples	25	5	30	1,500
Total	80	120	200	10,000

If funding is reduced, the schema and validation tools can be used independently of the registry.

Risks and maintenance

Adoption is the main uncertainty. Authors may find the schema burdensome, particularly if it duplicates metadata they already maintain. Early consultation and the indexing pilot will help us identify that burden. We will reuse existing metadata and revise the requirements in response to measured author effort before finalizing version 1.0.

Engine changes could break adapters or expose differences in test coverage. We will document supported versions and browser or server requirements, and use the example tutorials to detect regressions.

Public contribution and governance guidance will help others maintain the registry without relying on the two leads for every change. Entries will name maintainers and retain source links, validation dates, and status. Scheduled checks and an archival policy will help contributors address stale resources.

Success

Authors will be able to describe and check existing tutorials without changing tools. Learners will be able to find materials suited to their goals and experience, while instructors can assess lessons for reuse. Both will be able to compare requirements and check results across engines, with each tutorial’s review status clearly stated.

The table compares documented results with grant-end targets. We will update the baseline at grant start to include any completed pilot work.

Measure	Documented baseline	Grant-end target
Adapters with capability reports	0 demonstrated	3 initial adapters
Tutorials indexed through tutorialverse	0 demonstrated	5–10 across supported formats
External reviewers of schema, adapters, or tutorials	0 documented reviews	At least 3
Example tutorials tested in CI	0 demonstrated	4, collectively covering all adapters
Integration templates	0 demonstrated	3–4
Reusable tutorial-validation workflow	0 demonstrated	At least 1

Completion requires version 1.0 of the schema, R package, and registry. We must also publish and test the review and governance policies during the pilot, including procedures for handling stale resources.

Validation will apply only to the checks reported for each engine and environment. During the grant, we will track external submissions and review participation, alongside the effort authors spend supplying missing metadata. After release, outdated check results and broken links will guide maintenance, while package coverage will help us identify gaps in the registry.

Future work could add adapters for other tutorial engines or connect the registry to pkgdown, R-universe, and educational catalogs. Multilingual metadata, richer accessibility checks, and instructor collections remain outside the initial funded scope.

References

- Aden-Buie, Garrick, Barret Schloerke, J. J. Allaire, and Alexander Rossell Hayes. 2025. *learnr: Interactive Tutorials for R*. Released. <https://CRAN.R-project.org/package=learnr>.
- Castro, Leyla Jael, Patricia M. Palagi, Niall Beard, Teresa K. Attwood, and Michelle D. Brazas. 2023. “Bioschemas Training Profiles: A Set of Specifications for Standardizing Training Information to Facilitate the Discovery of Training Programs and Resources.” *PLOS Computational Biology* 19 (6): e1011120. <https://doi.org/10.1371/journal.pcbi.1011120>.
- Garcia, Leyla, Bérénice Batut, Melissa L. Burke, et al. 2020. “Ten Simple Rules for Making Training Materials FAIR.” *PLOS Computational Biology* 16 (5): e1007854. <https://doi.org/10.1371/journal.pcbi.1007854>.
- Geest, Geert van, Yann Haefliger, Monique Zahn-Zabal, and Patricia M. Palagi. 2024. “Using Glitr.org to Find, Compare and Re-Use Online Materials for Training and Education.” *PLOS ONE* 19 (12): e0308729. <https://doi.org/10.1371/journal.pone.0308729>.
- Handel, Andreas. 2025. *Quizzes and Exercises for Quarto Websites*. <https://www.andreashandel.com/posts/2025-02-25-exercises-in-quarto/>.
- Hiltemann, Saskia, Helena Rasche, Simon Gladman, et al. 2023. “Galaxy Training: A Powerful Framework for Teaching!” *PLOS Computational Biology* 19 (1): e1010752. <https://doi.org/10.1371/journal.pcbi.1010752>.
- learnr2: Interactive R Tutorials*. n.d. <https://github.com/PPBDS/learnr2>.
- Stagg, George. 2026. *quarto-live: WebAssembly Powered Code Blocks and Exercises for R and Python in Quarto Documents*. Released. <https://github.com/r-wasm/quarto-live>.
- The Carpentries. 2022. *Piloting Open Peer Review of Lessons*. <https://carpentries.org/blog/2022/02/carpentries-lab-lesson-review-pilot/>.